PROJECT PROGRESS REPORT

# Bioyond LabGenius S4

Autonomous Slide / Object-Handling Robot Cell

● Core grab pipeline live-validated on hardware

---

**Prepared for:** bekhruz.malikov@cephla.com

**Report date:** 2026-08-31

**Source:** full working directory (code, configs, calibration data, handover docs, git history)

**Repo snapshot:** commit `99ebe5d` — "Bioyond S4 autonomous ArUco grabbing — full project handover snapshot" (2026-07-28)

## 1 Executive Summary

The project goal is to reliably grab objects (initially microscope slides, currently ArUco-tagged lab items) at arbitrary deck positions using a Bioyond LabGenius S4 4-axis SCARA arm, driven either by an AI agent (via an MCP server), a human operator (via a Flask console), or a standalone headless script — all three sharing the same validated motion/vision/safety core.

As of the last recorded session (2026-07-28), the **full pipeline — find object → align → torque-guarded descend → grip → lift — has been live-validated on the physical robot multiple times**, including one fully autonomous run of the standalone ArUco grabber that survived two mid-run drop recoveries and finished with a verified grip and lift. Three independent camera and kinematic calibration systems were derived empirically from the live hardware (planar arm kinematics, claw-to-tool-center offset, and eye-in-hand camera mount), all backed by a self-checking calibration architecture (stamped constants + re-derivable anchors) built specifically because stale calibration constants caused real mis-grabs earlier in the project.

**What works today**
Connect → status → enable → run pipeline through either the MCP server, the operator console, or the standalone script. Camera-guided pick-and-place of ArUco-tagged objects at arbitrary (untaught) positions is validated. Both offline regression suites (96 checks in the console layer + the MCP server's own suite) are green.

**What's still open**
One grab was paused mid-motion at handover and needs to be parked/finished. A marker needs to be re-glued to a rigid part of its object. The MCP server doesn't yet have the console's automatic drop-recovery. See §9.

## 2 Timeline of Key Milestones

**2026-07-18** **First verified live motion.** A +20 mm Z `MoveL` executed with 0.000 mm lateral drift — baseline confidence in the BRCAPI TCP link, unit handling, and motion command path established.

**2026-07-19** **Eye-in-hand calibration effort begins.** Attempted to replicate Bioyond's own vision-guided grasp algorithm (detect → fine-tune → grasp) using hand-fitted pixel↔mm constants. Camera-to-jaw offset and scene Jacobian measured empirically for the first time.

**2026-07-20** **Vendor's official VisionTeach demo run to a successful grasp.** Rebuilt Bioyond's own C#/Halcon vision stack ( `S4-VisionDemo` ) from source to run locally; fixed path/locale/plumbing issues; root-caused repeated detection failures to **camera exposure** (taught config assumed a brighter cell). After correcting exposure, the demo detected the calibration board, converged its fine-tune loop, and lifted the board off the balance — confirmed on three independent cameras.

**2026-07-23** **Camera optics model corrected.** Discovered the hand-fitted focal length used since 07-19 was ~1.6× wrong (contaminated Jacobian measurement) and that the factory `CamInTool` calibration is a full 6-DOF pose (180° flip + 75.5° Z-rotation + translation), not a flat offset. Switched all pixel→robot math to the SDK-exact vendor path ( `mcp_teach.py` ).

**2026-07-24** **Anchor-free ArUco grabbing validated live at three blind positions.** The `aruco-grab` skill — solving an object's world pose from a marker with no taught point — worked with the claw lever-arm compensation applied.

**2026-07-25** **Eye-camera mount model fitted from 12 live poses**, RMS 0.26 mm — the calibration constant this project treats as its accuracy "heritage" benchmark for every future refit.

**2026-07-28** **Full-pipeline validation day.** Planar arm kinematics fitted to 0.00 mm residual across 8 poses (revealing the rotation center is offset from the world origin — a guard-logic correction). A bad camera fit was caught and reverted the same day via the stamp/anchor self-check. The complete pick pipeline (find → align → torque-guarded descend → grip → lift) ran live multiple times, including one **fully autonomous** run of `grab_aruco.py` that recovered from two dropped connections mid-run and still finished with a verified grip. Session ends with the arm paused mid-grab (see §10). Full project state committed to git as a handover snapshot.

Dates are drawn from in-code/in-doc session notes and the handover documents' own "as of" timestamps, not from git commit history — the repository was captured as a single handover snapshot commit, so git log does not carry a day-by-day trail.

## 3 System Architecture — Three Ways to Drive the Same Robot

All three interfaces sit on top of the same validated motion, safety, and calibration core, so a fix or lesson learned in one propagates to the others.

### 1. MCP Server LIVE `bioyond-s4-mcp/`

Exposes the robot as tools for an AI agent (Claude) to call in natural language — `connect`, `get_status`, `move_to_cartesian`, `move_gripper`, `camera_capture`, `gige_capture`, taught station jobs, etc. Talks BRCAPI (TCP/JSON) to the controller at `192.168.201.220:56788`. Registered via `.mcp.json`. Envelope-checks Cartesian moves server-side; requires human `operator_confirmed=true` for anything that bypasses the envelope or clears errors. Offline test suite passes against a fake controller (`tests/test_offline.py`). Gap: does not yet auto-recover a dropped mid-command connection the way the operator console does.

### 2. Operator Console LIVE `operator_ui/ (Flask, :5002)`

Human-in-the-loop web UI: the operator watches the eye-in-hand camera live and presses buttons for single, guarded steps — no autonomous agent decisions. Provides jogging, envelope editing, the full find→align→descend→grip→lift pipeline, camera and ArUco calibration workflows, a points/roles/trajectory canvas for multi-point pick-and-place workflows, and a live torque chart used to set the descent safety band. This layer is the most hardened: arrival verification (commanded vs. actual pose), automatic mid-command drop recovery (reconnect → settle → judge by pose, never blind-resend), and safe program restart logic all live here first. **96 offline regression checks** pass against a fake controller.

### 3. Standalone Grabber LIVE `grab_skill/grab_aruco.py`

A headless, no-browser, no-MCP Python script that runs the entire pipeline end-to-end by reusing the operator console's own service and sequence layers (so it inherits arrival verification and drop recovery automatically). This is the script behind the 2026-07-28 fully autonomous validation run.

### Claude Skills LIVE `.claude/skills/`

**grab-object** — the founding torque-guided pipeline: two context cameras find the object roughly, the eye-in-hand camera refines X/Y/Z, then a torque-monitored descent and soft grip complete the pick. Still authoritative for objects with no marker.
**aruco-grab** — the newer anchor-free pipeline: solves an ArUco-tagged object's world pose directly through the fitted camera model, compensates the measured 192 mm claw lever arm, and grips — validated live at three blind positions (2026-07-24).

### Vendor Reference & Context Pack REFERENCE `Vision_context/, Bioyond_S4_Agent_Context_Pack/`

The original Bioyond hardware/software manuals, raw BRCAPI protocol spec, controller Lua source, and Bioyond's own C# VisionTeach demo application (rebuilt and successfully run locally on 2026-07-20, see §2)

are kept for reference and were the starting point several of this project's own calibration routines were validated against.

### Exploratory / Adjacent Work EXPLORATORY `arducam/, camera_gui/`

A separate real-time detection/segmentation experiment (SAM 3, SAM 2.1, DINOv3 on a local RTX 4070) against the Arducam context cameras, plus a standalone camera testing GUI. Not yet wired into the production grab pipeline, which by explicit operator preference keeps object detection in the agent's own vision rather than local CV models.

## 4 Hardware & Connectivity

| Component | Detail | Status |
|---|---|---|
| Robot | Bioyond LabGenius S4 — 4-axis SCARA (J1 vertical ram, J2/J3 planar links, J4 wrist yaw); external axis E1 = gripper (E2 rotator present but unused) | **Commissioned** |
| Controller link | BRCAPI, TCP/JSON newline-framed, `192.168.201.220:56788`. Only ONE raw TCP client at a time — MCP server, operator console and Bioyond Works are mutually exclusive. | **Verified reachable today** |
| Eye-in-hand camera | GigE JHEM600GM-HM, mono 3072×2048, `192.168.201.89`, via MVS SDK — the precision sensor (~13-15 px/mm) | **Verified reachable today** |
| Context cameras | Two Arducam B0478 USB3 (1920×1080) for rough object localization before the eye-camera hand-off | **Unplugged late 07-28** |
| Gripper (E1) | Two comb plates, gap ≈ E1 + 60 mm; 28 = fully open (88 mm), 0 = closed (60 mm). Grip = torque < −100 AND position stall > 0.15 mm (free-torque baseline +150..+195) | **Characterized** |
| Per-joint torque telemetry | SysVarI 13-16 = J1-J4 normalized torque — only live when `main_hostpoint_trq.bin` is the published controller program | **Program-dependent** |

> **Connectivity check performed 2026-08-31:** this machine's Ethernet adapter ( `192.168.201.15` ) sits on the same subnet as the controller; ping to `192.168.201.220` succeeded (4/4, 0 ms), TCP port 56788 (BRCAPI) accepted a handshake, and the GigE camera address ( `192.168.201.89` ) responded. Network path to the robot is confirmed healthy as of this report.

## 5 Calibration Systems & Measured Machine Truth

Three independent geometric models were derived empirically from the live robot rather than trusted from vendor defaults — each was the direct cause of a real mis-grab before it was fixed, which is why the project subsequently built a self-checking calibration architecture around all three.

### 5.1 Planar arm kinematics

Fitted 2026-07-28 to **0.00 mm residual across 8 live poses**: `P = base + 200·e^(i·J2) + 200·e^(i·(J2+J3+180°))`, with **base = (126, 0)** — i.e. the arm's rotation center is *not* the world origin, which had been silently assumed by earlier reach-limit guards. Reach and joint-limit guards (J2 ≥ −93° soft margin, J4 ±150° preferred headroom, extension ≤385 mm from base) are now computed against this exact model.

### 5.2 Claw-to-tool-center offset ("claw ≠ TCP")

The gripper's actual pinch point hangs on a bracket **≈192 mm** from the tool center point the controller reports, at tool-frame offset (191.5, 5.6) mm. Every grab target must subtract this vector, rotated by the current wrist angle, before it's a valid TCP command. This was the project's founding calibration bug (see §7, lesson 1) and is now compensated automatically in the solver.

### 5.3 Eye-in-hand camera mount model

Fitted 2026-07-25 from 12 live poses to **0.26 mm RMS** — this figure is now the project's benchmark for judging whether any future re-calibration is healthy. The model maps a detected marker's pixel position + optical height directly to a world-frame pose, including camera mount offset, yaw misalignment, mirroring, and lens-tilt correction terms. A corrected understanding of the vendor's own factory calibration (true focal length, full 6-DOF `CamInTool` pose) underpins this fit — see the 2026-07-23 timeline entry.

### 5.4 Self-healing calibration architecture

Because a stale calibration constant after a camera refit once caused a 7.7 mm live miss, the calibration data is now versioned and cross-checked automatically:

- `calibration.json` stamps every derived constant (camera fit version, claw-arm model version) — the grabber **refuses to run** on mismatched stamps rather than silently using stale numbers.
- `anchors.json` holds ground-truth rows (pixel + optical height + verified grab pose) that can *re-derive* the claw arm and wrist-relation constants under whatever the current camera model is — this re-derivation runs automatically after every camera recalibration in the operator console.
- A multi-viewpoint spread check (agreement <2 mm across ≥2 solves) is the actual signal for camera-model health; anchor self-consistency alone can be fooled by a common-mode bias and does not by itself prove the model is correct.

## 6 Validated Capabilities

| Capability | Evidence | Status |
|---|---|---|
| Basic motion / link integrity | +20 mm Z MoveL with 0.000 mm lateral drift (2026-07-18) | **Validated** |
| Vendor vision-guided grasp (reference) | Bioyond's own VisionTeach demo detected the calibration board and completed a full grasp, confirmed on 3 cameras (2026-07-20) | **Validated** |
| Anchor-free ArUco grab at arbitrary position | Three blind live placements succeeded (2026-07-24) | **Validated** |
| Full pipeline: find → align → guarded descend → grip → lift | Run live multiple times on 2026-07-28 | **Validated** |
| Fully autonomous end-to-end run with self-recovery | `grab_aruco.py` completed one run that survived two mid-run connection drops and still finished with verified grip + lift (2026-07-28) | **Validated** |
| Mid-command TCP drop recovery | Reconnect → settle → judge-by-pose logic in the operator console/robot_service, never blind-resends | **Implemented & exercised live** |
| Offline regression coverage | 96 checks (operator console) + separate MCP server suite, both against a fake controller, both green | **Passing** |
| Network path to robot & cameras | Verified reachable from this workstation (2026-08-31) | **Verified** |

## 7 Safety Doctrine (Non-negotiable, Learned Live)

- A human must be at the E-stop for any motion; `operator_confirmed=true` requires a *fresh* human say-so, never once-per-session.
- A motion timeout or mid-command connection drop means **unknown physical state** — never resend; reconnect and judge by the actual pose read back.
- A "success" reply is not proof of a clean move — the controller has faulted mid-path while still returning success at least once (observed 16 mm + 47° off); only a pose match after the move counts.
- Torque band is a tripwire: the first breach stops the descent immediately; the arm never presses deeper "to confirm."
- Never close the gripper at a blocked depth; never (re)start the Lua program while the gripper is holding an object — a program start re-zeros E1 and slams the plates shut.
- Speed caps by phase: travel ≤30, approach ≤15-20, probing 5, lift 8-10.
- Joint-space (non-Cartesian) interpolation only for zero-translation rotations — its Cartesian path is otherwise unpredictable.

## 8 Testing

| Suite | Coverage |
|---|---|
| `operator_ui/tests/test_offline.py` | 96 checks against a fake controller: kinematics IK regression on live pose pairs, both live fault targets (reach/joint limits), safety guards, arrival verification, anchor derive/gate/restamp, all four grab-candidate geometry + span flags, per-object grip depth, the full find/align/descend/grip sequence chain, a complete simulated pick-and-place workflow with custom motion ordering, camera-fit recovery on synthetic scenes. |
| `bioyond-s4-mcp/tests/test_offline.py` | Full MCP tool surface against a fake controller emulating the deployed contract (fragmented TCP frames, meters-vs-mm pose units, string-typed params, invalid-job (-1) path) — no robot required. |

Both suites are treated as the project's living contract — the handover documentation explicitly calls for a new regression test to be added for every future live lesson learned, and for both suites to stay green before trusting new hardware runs.

## 9 Hard-won Lessons

Each of the following cost real debugging time on the live hardware and is preserved so it isn't relearned.

1. Claw ≠ TCP (192 mm arm) — the founding calibration bug.
2. Stale constants survive a camera refit unless explicitly re-derived (cost a 7.7 mm live miss) → stamps + anchors.
3. Stale motion acknowledgements / drops → arrival verification + settle-and-judge-by-pose.
4. An early 3-pose kinematic fit was a geometric phantom (too narrow a sector) — fits now require well-spread poses.
5. Origin-centered reach guards were wrong; the arm's true rotation base sits at (126, 0).
6. Off-center or off-viewing-band vision solves bias by millimeters → enforce a center-of-frame rule.
7. Printed marker scale can differ from its nominal size → verify every new marker sheet with a two-height check.
8. A joint's stationary torque reading is not comparable to its post-step reading → use a trend rule, not an absolute one.
9. Calibrating against a marker resting on loose/unfixed contents silently poisons the camera fit.
10. A marker on loose contents is a drifting reference in both Z and XY, not just at calibration time.
11. Two visible markers or a reflection ~190 mm apart usually means one sighting is not the intended object.
12. Switching the controller from manual to auto silently drops the servo enable — re-enable before starting the program.
13. E1 (gripper) re-zeros on every program start — never restart the controller program while holding an object.

## 10 Physical State at Last Handover & Open Items

> **Arm was left paused mid-grab** at TCP (317.43, −325.78, Z 63.2, Rz 86.55) — south approach over the object on the deck, grab point (323.44, −134.46), **claw OPEN at 28**, descent baseline half-taken. The controller connection was also held by another client at handover time. **First action on resuming work should be to park the arm** (lift to Z ≥ 280, close the claw) before anything else.

| # | Area | Item |
|---|---|---|
| 1 | Grab completion | Finish or redo the paused deck grab — the 3-viewpoint solve is already locked with 0.22 mm spread; target coordinates recorded above. |
| 2 | Fixture reliability | Glue the current object's marker to its rigid rim/frame — it currently sits on a loose pile and drifts in Z/XY, which is the single largest remaining source of grab error. |
| 3 | MCP server parity | Port the operator console's mid-command drop-recovery logic into the MCP server (currently console-only). |
| 4 | Workflow engine | Support per-leg motion orders in multi-point workflows (currently one global motion order for the whole run). |
| 5 | Object modeling | The current object is really a two-body assembly (a loose tray inside an adapter plate) — earlier grips sometimes clamped the wrong body; each body deserves its own marker/id. |
| 6 | UI | No explicit E-stop status indicator in the console header (currently folded into the general errors chip). |
| 7 | Safety policy | The claw-window/span rejection check is currently warning-only by explicit operator decision — worth revisiting before any unattended operation. |

## 11 Repository Inventory (Reference)

| Path | Contents |
|---|---|
| `bioyond-s4-mcp/` | MCP server: `server.py` (tools), `robot.py` (hardened TCP client), `hikcam.py` (GigE), offline test suite |
| `operator_ui/` | Flask human-in-the-loop console, sequences, robot/camera services, points/trajectory UI, config JSONs, offline test suite |
| `grab_skill/` | Vision solvers, calibration/anchor data, torque probing, the standalone `grab_aruco.py` grabber |
| `.claude/skills/` | `grab-object` and `aruco-grab` Claude Agent skills |
| `Bioyond_S4_Agent_Context_Pack/` | Vendor manuals (hardware, software, protocol, integration), controller Lua source, Python reference clients, agent project brief |
| `Vision_context/` | Vendor's own C#/Halcon VisionTeach demo application and calibration data, rebuilt and run locally |
| `arducam/` | Exploratory real-time detection (SAM 3 / SAM 2.1 / DINOv3) against the context cameras |
| `camera_gui/, newcam_images/` | Camera testing utility and captured reference frames |
| `CLAUDE.md` | Operational contract for any agent working in this repo — quick-start sequence, tool table, safety rules, controller contract |
| `HANDOVER_MCP_SKILLS.md / HANDOVER_OPERATOR_UI.md` | The two engineering handover documents this report is substantially drawn from |

Generated from the working directory at `C:\Users\Cephla\Downloads\Bioyond_Cephla\Bioyond_Cephla` on 2026-08-31. Compiled by Claude from CLAUDE.md, both handover documents, README files, session notes, config/calibration JSON, and git history — not an independent re-verification of the robot's current physical state.